Leveraging Sparsity and Quantization in DNN Vector Algebra

Poorna Ravuri
Department of Computer Engineering
San Jose State University
San Jose, California
poorna.ravuri@sjsu.edu

***Abstract*—Artificial Intelligence (AI), particularly Deep Neural Networks (DNNs), has achieved remarkable accuracy in various cognitive tasks. However, DNNs demand significant computational resources, leading to challenges in memory usage, computation time, and power consumption, especially for deployment on mobile or embedded devices. Quantization methods, notably fixed-point quantization, offer promising avenues to reduce these demands, but fixed bit-width can compromise accuracy. This research investigates Bit-Slicing Vectors (BSVs), a data structure enabling fine-grained precision control and effective sparsity utilization, for DNN computations. We propose and evaluate BSV-based algorithms for pivotal vector operations: dot product, addition, and multiplication. Our experiments on CPU platforms compare BSV performance against standard PyTorch functions and C++ implementations using both randomly generated data and extracted BERT model weights. Initial results indicate that while BSVs can offer advantages in memory footprint, their computational overhead for operations like dot product and multiplication on CPUs needs further optimization to compete with conventional methods in terms of speed. Nonetheless, the flexibility of BSVs in managing precision and sparsity warrants continued investigation, especially for resource-constrained environments and specialized hardware.**

I. Introduction

Artificial Intelligence (AI) has taken the world by storm, with research and applications powered by AI growing exponentially each year. The true potential of AI was realized with the development of Deep Neural Networks (DNNs) in 2012 [1], which coincided with the widespread adoption of Graphics Processing Units (GPUs)—devices that offer massive parallel processing capabilities. Typically, DNNs require GPUs or large parallel processing clusters for training. However, during inference, they are deployed as static models on power-constrained devices like mobile phones or embedded systems that cannot directly support the high computational demands of DNNs. To address this, both software and hardware optimizations have been proposed, with quantization being one of the most widely used methods. Quantization helps reduce compute, memory, and network demands, yet this technique, involving low bit-width, can lead to a significant loss in prediction accuracy. To mitigate this, our project aims to explore the use of Bit-Slicing Vectors (BSVs) as a potential data structure to store DNN vectors, providing better control over quantization while minimizing accuracy loss. By focusing on tensor operations such as dot products, summation, aggregation, and threshold functions—which significantly contribute to the processing load—we aim to enhance model performance in machine learning systems. We propose a novel approach by using a hybrid compression mechanism on the model weights, performing the required tensor operations, and comparing the times between the compression approach and the usual tensor operations method. We are testing this approach on BERT - Bidirectional Encoder Representation from Transformers model. We are comparing the performance of key tensor operations (dot product, addition, and multiplication) using standard PyTorch functions, optimized C++ vector implementations, and our BSI-based approach.

II. Motivation

DNN algorithms are superior but come at a higher computational cost. This is acceptable during training; however, during inference, the developed models are often deployed on resource-constrained devices as static models, which may or may not require extensive computational power. Nonetheless, deployment can be challenging on devices such as mobile phones and embedded systems. Fixed-point quantization has been proposed to enable these models to function effectively on such devices [2] [3] [4]. However, this is not a novel method. The primary issue with fixed-point quantization method is the fixed bit-width, which can significantly impair model classification accuracy. The fixed low bit-width DNNs may suffer losses in classification accuracy due to quantization errors. Similarly, for DNN training, the training dataset sizes grow faster than advancements in DNN hardware [5]. A software solution to this problem, could allow for flexible bit-width selection, that can be combined with data compression. It could be deployed on processing units from different vendors with various architectures while increasing energy efficiency and processing speeds. This has motivated us to use bit-slicing vectors (BSVs) as a potential data structure that allows us to control the precision of each vector operation by selecting the number of vector slices to be processed. A similar approach was employed in implementing dynamic quantization based on query parameters in similarity queries [6], and the encouraging results there have motivated us to apply a similar technique in DNN tensor operations.

III. Related Work

In deep neural networks (DNNs), the model performs both forward and backward passes. During a forward pass, the inputs will propagate through the network to generate output predictions based on the current weights. During a backward pass, the predicted output is compared with ground truth to update the model parameters, using gradient descent algorithm, and this is called as backpropagation. Backpropagation [7] algorithm will try to reduce loss by updating model parameters, and the key component in this algorithm is calculating the gradient. Gradient calculation will require higher bit precision to maintain numerical stability that would help the model to converge to an optimal solution, and this would require a lot of computation. This is acceptable during training, but during inference, the model could be deployed on computationally constrained devices. To tackle this, lower bit precision is used during inference. This will compromise the model performance, but a numerical accuracy loss is acceptable as long as it is in the acceptable range. These optimizations were classified into software optimization and hardware classification. To test the algorithm, we used BERT (Bidirectional Encoder Representations from Transformers) as our baseline model.

A. Software Optimization

Software optimizations will involve making changes to the software aspect of the system which will essentially result in the improved compute performance. These optimizations can be further divided into DNN Compression and Model Optimization.

1) DNN Compression: DNN compression is a process of reducing the complexity of the model without loss in its performance, which will make the model lightweight. Common techniques in DNN compression include network pruning, quantization, and knowledge distillation [8].

Sparse quantized representation (SpQR) [9] is a method that involves qunatization. This method involves the identification of outlier weights which are causing high erros, and we store them in higher bit precision. To find these outlier weights, a small set of calibration inputs is run through the model. The sensitive weights are identified by using an equation which will identify the difference between orginal predictions and quantized weight predictions. Non-outlier weights are quantized to 3-4 bit precision while the sensitive weights are qunatized to 16-bit precision.

2) Model Optimization: Model optimization involves making changes to the model architecture which will result in quicker computation and lower energy requirement. A method proposed in quantization networks [10] is to use a non-linear transform function that can be applied to both weights and activations in DNNs. This method uses a Temperature factor T which will help in finding the right balance between model learning capacity and quantization gap. Quantization network outperformed the existing state-of-the-art methods on object classification and detection. Another proposed method is to use an optimization technique that involved building an Energy Interface Engine (EIE) [11] that will prune redundant layers and have multiple connections with same weights such that DNN can fit in on-chip SRAM. This engine results in energy saving, sparsity exploitation, and this is performing better than non compressed DNNs. One drawback of this method is that it only exploited the sparsity of fully connected layers (FNN).

B. Hardware Optimization

Standard deep learning applications use 32-bits of floating point precision during training. Optimization is typically done during inference, however, research has shown that lower numerical precision using 16-bit and 8-bit multipliers can still achieve similar accuracy levels while using lesser memory and performing higher number of operations per second [12]. As per [13] 16-bit fixed point numbers can be used to train state-of-the-art neural networks by employing stochastic rounding during fixed-point computations. [14] further reduced the number of bits required for training by introducing a new 8-bit floating point format (FP8) using chunk-based computations and stochastic rounding that achieved similar accuracy to full-precision training. It is even more challenging to create optimized mobile deep learning applications because of the limited computations resources available compared to desktop or server machines. [15] proposed a unified neural processing unit for mobile DNN applications that allows for fully variable precision from 1 to 16 bit with the help of lookup table-based bit-serial processing element.

C. BERT - Bidirectional Encoder Representations from Transformers

BERT (Bidirectional Encoder Representations from Transformers) is a large language model developed by leveraging the Transformer architecture, specifically utilizing the encoder block [16]. Since its release, BERT has revolutionized various natural language processing (NLP) tasks and propelled research in the field. The key innovation introduced by BERT is its deep bidirectional understanding of language context, enabling it to capture context from both left and right directions in text [17]. Unlike previous models that processed text either from left-to-right or combined left-to-right and right-to-left training (e.g., ELMo, GPT), BERT is designed to pre-train deep bidirectional representations by jointly conditioning on both left and right context in all layers. BERT consists of multiple layers of self-attention mechanisms and feedforward neural networks, allowing it to capture complex dependencies in the text. The base model, BERT-base, contains 12 transformer layers (also known as encoder blocks), 768 hidden units, and 12 self-attention heads, totaling around 110 million parameters. BERT-large scales this up to 24 layers, 1024 hidden units, and 16 attention heads, amounting to approximately 340 million parameters. BERT is pre-trained on two unsupervised tasks:

- Masked Language Modeling (MLM): Randomly masks a percentage of the input tokens and predicts the masked tokens based on the context provided by the unmasked

tokens on both sides. This enables the model to learn deep bidirectional representations.

- Next Sentence Prediction (NSP): Predicts whether a given pair of sentences appears consecutively in the original text, allowing the model to understand sentence relationships.

These pre-training tasks enable BERT to learn rich linguistic representations that can be fine-tuned for various downstream tasks with minimal task-specific architecture modifications. The success of BERT has spurred a wave of research into transformer-based models and pre-training techniques, leading to numerous variants and extensions, such as RoBERTa [18], ALBERT [19], and XLNet [20].

IV. Methodology

Our research aims to optimize the computational time required by the model during training and inference. To achieve this, we draw upon principles similar to those in [21] for leveraging data characteristics like sparsity and employing compression, adapting these for vector operations within DNNs. The primary mechanism in BERT and other transformer-based models is the calculation of attention scores, detailed in [16]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{1}$$

where Q, K, and V are Query, Key, and Value matrices, respectively. This formulation highlights the importance of matrix multiplications (which are sequences of dot products), element-wise operations, and aggregations. We propose using our own BSI-based algorithms for these fundamental vector operations—specifically dot product, BSI vector summation, and element-wise multiplication—to explore potential performance benefits compared to standard library functions. The core of our approach lies in Bit-Slicing Vectors (BSV).

A. Bit Slicing Vectors

A vector is an ordered array of numbers, where each number is identifiable by its index [22]. Bit-Sliced Indices (BSI) [6] provide a method to encode numerical attributes using bit-wise operations, often for efficient querying in high-dimensional data. In BSI, the absolute value of a number is typically stored in a hybrid bitmap (compressed or uncompressed), while its sign is stored in a separate bitmap. An existence bitmap tracks valid entries. This paper adapts the BSI concept to represent DNN vectors as Bit-Sliced Vectors (BSVs), aiming to control precision and leverage sparsity.

B. Dot Product Operation

The dot product of two vectors yields a scalar sum of their pairwise elemental products. In BSVs, encoding elements in binary allows for an optimized dot product calculation (Algorithm 1). The process involves performing AND operations on corresponding bit-slices to find shared '1's. The count of these shared bits, weighted by their positional significance (2^k) and adjusted for the signs of the original numbers, contributes to the final scalar result.

Algorithm 1 Dot Product of two BSVs, A and B, yielding $C = A \cdot B$

Require: BSV A, BSV B (with A_{slices}, B_{slices} for magnitudes; A_{sign}, B_{sign} for signs)
Ensure: Scalar dot product C

```
Initialize C ← 0
for i ← 0 to numSlices(A) − 1 do          ▷ Iterate through slices of A (magnitude)
    for j ← 0 to numSlices(B) − 1 do      ▷ Iterate through slices of B (magnitude)
        common_bits ← A_slices[i] ∧ B_slices[j]      ▷ Shared set bits
        prod_signs_bitmap ← A_sign ⊕ B_sign          ▷ 1 if signs differ, 0 if same
        positive_terms_mask ← ¬prod_signs_bitmap     ▷ Mask for positive products
        negative_terms_mask ← prod_signs_bitmap      ▷ Mask for negative products
        R_pos ← common_bits ∧ positive_terms_mask
        R_neg ← common_bits ∧ negative_terms_mask
        weight ← 2^(i+j)                             ▷ Weight factor; see text for details
        C ← C + (numberOfOnes(R_pos) · weight)
        C ← C − (numberOfOnes(R_neg) · weight)
    end for
end for
return C
```

C. BSI Vector Summation Operation

This operation, detailed in Algorithm 2, computes the sum of all numerical values held within a single BSI vector. It iterates through each bit-slice, counts the number of set bits ('1's), multiplies this count by the slice's positional weight (2^i for the i-th slice, assuming 0-indexed slices for magnitude), and adds it to an accumulating sum. This is useful for aggregation tasks. Note that signs of the numbers within the BSI vector must also be accounted for if the vector contains mixed-sign numbers; Algorithm 2, as written, sums magnitudes based on direct bit counts. A full signed sum would incorporate the sign bitmap, as handled by the underlying C++ implementation.

D. BSI Element-wise Vector Multiplication Operation

Element-wise multiplication creates a new vector where each element is the product of corresponding elements from two input vectors. For BSVs, this involves multiplying element magnitudes (represented by their bit-slices) and determining the sign of each resulting product (typically by XORing the signs of the input elements). Algorithm 3 outlines this. The multiplication of magnitudes often mimics binary long multiplication, adapted for the sliced representation, and may be performed word by word from the underlying bitmaps.

E. Approach

Our experimental approach involved several steps. First, we extracted weights from both pre-trained and fine-tuned

Algorithm 2 BSI Vector Summation: Sum of elements in a BSI vector A

Require: BSI vector A (composed of N bit-slices $A_0, \ldots, A_{N-1}$ for magnitudes, and A_{sign} bitmap)
Ensure: Scalar sum S

```
Initialize S ← 0
for i ← 0 to N − 1 do ▷ Iterate through each magnitude bit-slice
    slice_contribution ← 0
    onesCount ← numberOfOnes(A_i)   ▷ Count set bits in i-th magnitude slice
    S ← S + onesCount · 2^i   ▷ Add weighted count (magnitudes)
end for
return S   ▷ Actual signed sum logic in C++ code
```

Algorithm 3 BSI Element-wise Vector Multiplication: $C = A \odot B$

Require: BSI vectors A, B (same length, with magnitude slices and sign bitmaps A_{sign}, B_{sign})
Ensure: BSI vector C (representing V_C where $V_{C_k} = V_{A_k} \cdot V_{B_k}$)

```
Initialize C as an empty BSI structure.   ▷ Max slices for C_mag in text
C_sign ← A_sign ⊕ B_sign   ▷ Result sign bitmap
for each corresponding pair of elements (val_A, val_B) conceptually in A, B do
    Let a_slices be bit-slices of val_A's magnitude.
    Let b_slices be bit-slices of val_B's magnitude.
    product_magnitude_slices ← MultiplyMagnitudesAsBitSlices(a_slices, b_slices)   ▷ Binary multiplication logic
    Append product_magnitude_slices to C's magnitude slices for the current element.
end for
Assign C_sign as the sign bitmap for C.
Set other metadata for C (e.g., existence bitmap from A or B).
return C
```

(on the IMDB dataset) BERT models, flattening these tensor weights into 1-D arrays. We then compared three methods for performing key vector operations—dot product, BSI vector summation (Algorithm 2, which was labeled 'Addition' in our comparative tests in Figure 1), and element-wise multiplication:

1) Native PyTorch functions (e.g., `torch.dot`, element-wise operators).
2) Our BSI-based implementations, which utilize a hybrid compression mechanism as described in [21].
3) Standard C++ vectorized implementations as a performance baseline.

Floating-point weights were scaled to integers before BSI conversion, using a precision factor. The number of bits required for the magnitude of a scaled integer x ($x \neq 0$) is $\lfloor \log_2(|x| + 1) \rfloor + 1$. An additional bit (or separate bitmap) is used for the sign. This dictates the BSI structure's depth for magnitudes. For each operation and method, we measured execution times (averaged over multiple runs) and memory usage. Error percentages relative to PyTorch results were also noted for BSI operations to check numerical fidelity. The performance visualizations are presented in Figures 1 and 2.

V. Dataset

For testing the algorithm, we have used both the pre-trained version of the BERT model and the trained version of BERT. For training the model, the IMDB [23] dataset is used. This is an open-source dataset available on hugging face. This is a data set for binary sentiment classification that contains substantially more data than previous benchmark data sets. It provides a set of 25,000 highly polar movie reviews for training and 25,000 for testing. There are additional unlabeled data for use as well. The data set can be directly loaded using hugging face modules and will be downloaded or stored on the system as a dictionary. We can fetch the train and test lists using the 'train' and 'test' keys. It consists of text and label, where text is a string feature and label is a classification label with values 0 and 1.

VI. Evaluation

Our evaluation focused on comparing the performance of dot product, BSI vector summation (referred to as 'Addition' in Figure 1), and element-wise multiplication operations. We benchmarked our BSI-based methods against standard PyTorch tensor functions and optimized C++ vector operations. The primary metrics were execution time and memory usage, gathered on the hardware platforms detailed below. Error percentages were also monitored to ensure the numerical accuracy of BSI operations remained within acceptable limits compared to high-precision baselines.

A. Hardware & Software Stack Used

1) Hardware - 1 Used (CPU Platform): Primary CPU-based experiments were conducted on a Lenovo ThinkPad X1 Carbon (5th Generation):

- Processor: Intel® Core™ i7-7600U CPU @ 2.80GHz (4 cores)
- Memory: 16 GB RAM
- Graphics: Integrated Intel® HD Graphics 620
- Storage: 1 TB SSD
- OS: Ubuntu 22.04.4 LTS (64-bit), GNOME 42.9 (Wayland)

2) Hardware - 2 Used (HPC Platform): For scalability and potential acceleration studies, access to a High-Performance Computing (HPC) system was utilized:

- Graphics Hardware: Four NVIDIA A100-PCIE-40GB GPUs
- GPU Driver: 550.54.15, CUDA Version: 12.4
- Per-GPU Memory: 40 GB, Power Limit: 250 W

3) Software Used: The following software stack supported our evaluations:

- IDE: Visual Studio Code v1.95.3
- Language/Compiler: Python $\geq$ 3.8, C++ (g++ $\geq$ 11.4.0)
- Libraries: PyTorch v2.4.0, Hugging Face Transformers v4.44.2

B. Performance with Randomly Generated Data (on CPU)

Figure 1 illustrates the performance of multiplication, dot product, and BSI vector summation (labeled 'Addition' in the figure, corresponding to Algorithm 2) using randomly generated input vectors, each of 10 million elements. These tests were run on the CPU platform (Hardware-1) and compare our BSI-10M Vector approach against a 'Standard Vector' baseline (representing PyTorch or a similar highly optimized library). Performance was assessed across three distinct element value ranges: [0, 100], [0, 1000], and [0, 2^{16}], simulating different data magnitudes and potential sparsity implications.

Regarding execution time for multiplication, the BSI method was observed to be slower than the standard vector across all tested value ranges, with this time difference increasing as the magnitude of values increased. For dot product, BSI also exhibited significantly slower execution times. In contrast, for BSI vector summation (Addition), the BSI approach was slower for the smaller value ranges but demonstrated a slight speed advantage at the highest value range of [0, 2^{16}].

In terms of memory usage, the BSI method for multiplication showed varied results: it used slightly less memory at the [0, 100] range, was comparable for [0, 1000], and consumed slightly more memory for the [0, 2^{16}] range. However, for both dot product and BSI vector summation (Addition), BSI consistently consumed less memory than the standard vector implementation across all tested value ranges, highlighting a potential benefit for memory-constrained scenarios.

C. Performance with Extracted BERT Weights (on CPU)

Figure 2 focuses on dot product performance using actual weights extracted from a BERT model previously trained on the IMDB dataset. These vectors also had a length of 10 million elements. The comparison here is specifically between our BSI approach and a C++ vector implementation, with tests conducted on the CPU platform (Hardware-1) across the same three value ranges as the random data experiments.

The results presented in Figure 2 clearly indicate that the BSI-based dot product was consistently slower in execution time than the C++ vector baseline for all tested value ranges. The performance disparity was particularly pronounced for the [0, 2^{16}] range, where the BSI approach took substantially longer. This suggests that, for these specific conditions on the CPU, the computational overhead associated with the BSI method (even when incorporating hybrid compression) for dot products on realistic DNN weights is significant when compared to a more direct C++ implementation.

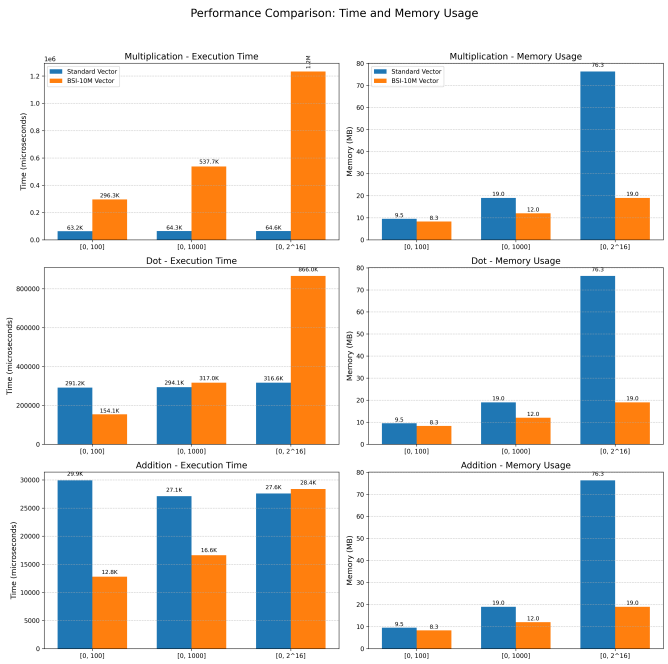

Fig. 1. Performance (Execution Time in μs, Memory in MB) of Multiplication, Dot Product, and BSI Vector Summation (Addition) with randomly generated 10M-element vectors on CPU (Hardware-1). Compares Standard Vector vs. BSI-10M Vector across different element value ranges.

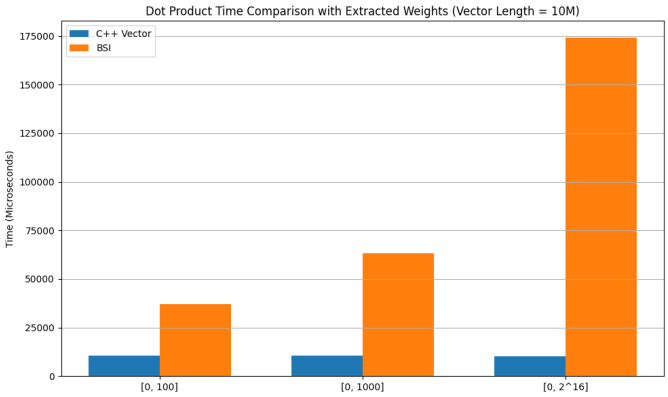

Fig. 2. Dot product execution time (microseconds) comparison: C++ Vector vs. BSI, using 10M-element vectors from extracted BERT weights (trained on IMDB data) on CPU (Hardware-1), across different element value ranges.

VII. Conclusion

Our research investigated the application of Bit-Slicing Vectors (BSVs) to core vector operations in Deep Neural Networks—specifically dot product, BSI vector summation, and element-wise multiplication. Through evaluations on a CPU platform, comparing BSV-based methods against standard PyTorch functions and C++ implementations with both randomly generated data (Figure 1) and actual weights extracted from a BERT model (Figure 2), several key observations emerge.

The CPU-based experiments reveal a consistent trend: BSVs offer notable advantages in memory consumption for operations such as dot product and BSI vector summation across

various data value ranges. This characteristic is particularly relevant for deploying large models in environments with limited memory. However, this memory efficiency came at a cost in computational speed on the CPU platform. Our BSI implementations for dot product and multiplication were generally slower than their conventional counterparts. For BSI vector summation (labeled 'Addition' in tests), a slight speed advantage for BSI was observed only at the largest tested value range with random data. When using extracted BERT weights for dot product operations on the CPU, the BSI method was markedly slower than an optimized C++ vector implementation, especially as the data value range increased.

These findings suggest that, on the tested CPU architecture, the overhead associated with managing the BSI data structure and performing bitwise manipulations, even with the use of hybrid compression, can outweigh the direct computational efficiency of standard library operations for these tasks. While the flexibility in precision and memory savings are valuable, the current BSI performance on CPUs indicates that significant speedups require either further substantial algorithmic optimization or leveraging hardware architectures more amenable to bit-parallel processing.

The path forward should focus on overcoming these computational limitations. Key opportunities include:

- Comprehensive benchmarking on massively parallel hardware, such as the HPC-grade GPUs (Hardware-2), to determine if BSI's inherent parallelism can be translated into significant execution speedups not seen on CPUs.
- Further refinement of BSI C++ implementations and the hybrid compression strategy to reduce overhead.
- A detailed study of the interplay between very low bit-width BSI representations (e.g., 2-bit to 5-bit) and their impact on both system performance and end-task model accuracy.

In conclusion, while BSVs present a compelling paradigm for memory-efficient and precision-flexible vector algebra, achieving competitive computational speed against highly optimized libraries on standard CPUs remains a challenge. The true potential of BSVs for accelerating DNN computations may lie in specialized hardware environments or through continued breakthroughs in bitwise algorithm design.

VIII. Future Work

Building on our current findings, future research will target several key areas to enhance the viability of Bit-Slicing Vectors for DNN computations:

- **Exploiting Parallel Architectures:** The most critical next step is a comprehensive performance analysis of BSI operations on GPUs (like the NVIDIA A100s in Hardware-2) and potentially other parallel processors (e.g., FPGAs). This is essential to quantify the speedups achievable when BSI's inherent parallelism is matched with suitable hardware, which was not the focus of the CPU-centric results presented herein.
- **Advanced Algorithmic Optimization:** We will continue to refine the BSI C++ algorithms, focusing on reducing instruction counts, improving cache utilization, and exploring more sophisticated hybrid compression strategies tailored for specific data distributions found in DNN weights and activations.
- **Low-Precision Studies:** A detailed investigation into the efficacy of very low-precision BSIs (e.g., 2-bit to 5-bit representations) is planned. This includes quantifying the trade-offs between computational/memory savings and any resultant loss in model accuracy for tasks like those performed by BERT.
- **End-to-End Model Integration:** We aim to integrate our optimized BSI vector operations into a complete DNN inference pipeline (e.g., within the BERT attention mechanism) to assess system-level performance impacts and identify bottlenecks beyond individual operations.
- **Open-Source Contribution:** To foster broader research and application, we plan to develop and release an open-source library of our BSI tools, potentially with Python bindings for easier integration with frameworks like PyTorch.

References

[1] G. E. H. Alex Krizhevsky, Ilya Sutskever, "Imagenet classification with deep convolutional neural networks," *Advances in Neural Information Processing Systems (NIPS)*, 2012.

[2] H. Tann, S. Hashemi, R. I. Bahar, and S. Reda, "Hardware-software codesign of accurate, multiplier-free deep neural networks," in *2017 54th ACM/EDAC/IEEE Design Automation Conference (DAC)*. IEEE, 2017, pp. 1–6.

[3] E. Park, D. Kim, and S. Yoo, "Energy-efficient neural network accelerator based on outlier-aware low-precision computation," in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2018, pp. 688–698.

[4] S. Jain, S. Venkataramani, V. Srinivasan, J. Choi, P. Chuang, and L. Chang, "Compensated-dnn: energy efficient low-precision deep neural networks by compensating quantization errors," in *2018 55th ACM/ESDA/IEEE Design Automation Conference (DAC)*. IEEE, 2018, pp. 1–6.

[5] C.-J. Wu, R. Raghavendra, U. Gupta, B. Acun, N. Ardalani, K. Maeng, G. Chang, F. Aga, J. Huang, C. Bai *et al.*, "Sustainable ai: Environmental implications, challenges and opportunities," *Proceedings of Machine Learning and Systems*, vol. 4, pp. 795–813, 2022.

[6] G. Guzun, J. Tosado, and G. Canahuate, "Slicing the dimensionality: Top-k query processing for high-dimensional spaces," in *Transactions on Large-Scale Data-and Knowledge-Centered Systems XIV*. Springer, 2014, pp. 26–50.

[7] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.

[8] R. Mishra, H. P. Gupta, and T. Dutta, "A survey on deep neural network compression: Challenges, overview, and solutions," 2020.

[9] T. Dettmers, R. Svirschevski, V. Egiazarian, D. Kuznedelev, E. Frantar, S. Ashkboos, A. Borzunov, T. Hoefler, and D. Alistarh, "Spqr: A sparse-quantized representation for near-lossless llm weight compression," 2023.

[10] J. Yang, X. Shen, J. Xing, X. Tian, H. Li, B. Deng, J. Huang, and X.-s. Hua, "Quantization networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2019.

[11] S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally, "Eie: Efficient inference engine on compressed deep neural network," *ACM SIGARCH Computer Architecture News*, vol. 44, no. 3, pp. 243–254, 2016.

[12] A. Rodriguez, E. Segal, E. Meiri, E. Fomenko, Y. J. Kim, H. Shen, and B. Ziv, "Lower numerical precision deep learning inference and training," *Intel White Paper*, vol. 3, no. 1, p. 19, 2018.

[13] S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan, "Deep learning with limited numerical precision," in *International conference on machine learning*. PMLR, 2015, pp. 1737–1746.

[14] N. Wang, J. Choi, D. Brand, C.-Y. Chen, and K. Gopalakrishnan, "Training deep neural networks with 8-bit floating point numbers," *Advances in neural information processing systems*, vol. 31, 2018.

[15] J. Lee, C. Kim, S. Kang, D. Shin, S. Kim, and H.-J. Yoo, "Unpu: An energy-efficient deep neural network accelerator with fully variable weight bit precision," *IEEE Journal of Solid-State Circuits*, vol. 54, no. 1, pp. 173–185, 2018.

[16] A. Vaswani, N. M. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*. Neural Information Processing Systems Foundation, Inc., 2017.

[17] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, vol. 1. Association for Computational Linguistics, 2019, pp. 4171–4186.

[18] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.

[19] Z. Lan, M. Chen, S. Goodman, K. Gimpel, P. Sharma, and R. Soricut, "ALBERT: A lite BERT for self-supervised learning of language representations," *arXiv preprint arXiv:1909.11942*, 2019.

[20] Z. Yang, Z. Dai, Y. Yang, J. Carbonell, R. Salakhutdinov, and Q. V. Le, "XLNet: Generalized autoregressive pretraining for language understanding," in *Advances in Neural Information Processing Systems*, vol. 32. Neural Information Processing Systems Foundation, Inc., 2019, pp. 5753–5763.

[21] G. Guzun and G. Canahuate, "Hybrid query optimization for hard-to-compress bit-vectors," *The VLDB Journal—The International Journal on Very Large Data Bases*, vol. 25, no. 3, pp. 339–354, 2016.

[22] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.

[23] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts, "Learning word vectors for sentiment analysis," in *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*. Portland, Oregon, USA: Association for Computational Linguistics, June 2011, pp. 142–150. [Online]. Available: http://www.aclweb.org/anthology/P11-1015